QCon
全球软件开发大会

InfoQ 技术大会

「AI 时代架构往哪走？」

——快手 AI 原生架构如何演进？

AI 越用越快，但没有统一规划就会越用越乱 → 先夯地基，业务才能在 AI 时代跑得 更快、更稳、更远

刘中兵 / 快手 架构治理团队 & AI 架构演进负责人



QCon 全球软件开发大会 · 上海 2026

面向AI架构师 + 技术管理者 + AI应用工程师 | 45 分钟

讲师介绍 & 本场定位

分享讲师 刘中兵

- 架构治理 · 顶层全局AI架构视角
- 主导公司级AI原生架构演进规划
- 主导快手架构成熟度体系与治理



内容聚焦

- **看清全局**：集合全公司架构师和行业视角，看清**AI应用全面爆发带来的全局痛点**
- **看清路线**：首次提出AI架构演进蓝图规划，看清**AI时代架构与传统架构的共生关系**
- **看清元素**：闭环看清楚AI架构治理的元素，覆盖**AI代码生成、CLI、Skill、Agent、安全、Token、模型、引擎等关键元素**
- **看清打法**：为了避免技术治理会影响业务，制定**如何抓重点、如何落地的打法原则**

适合人群

- **AI架构规划**：AI架构师、技术管理者
- **AI落地应用**：AI应用工程师、AI平台建设者

本场主线 & 你关心的 5 个问题

一句话主线：AI 越用越快，但没有统一规划就会越用越乱 —— 先夯地基，业务才能在 AI 时代跑得更快、更稳、更远

①问	②问	③问	④问	⑤问
AI把架构改成什么样子?	我们遇到了哪些问题，公司架构师看到了什么问题?	未来的蓝图规划长什么样子?	未来如何落地AI架构演进规划?	下一步应该先做什么、后做什么？各方需要做什么?

带着这 5 个问题听 —— 下面五个篇章逐一回答

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 行业趋势与认知：AI 把架构改成什么样子？业界是怎么做的？

02 公司现状与痛点：公司遇到了什么问题？挑战在哪里？

03 AI架构蓝图规划：AI 原生架构应该怎么建？

04 AI架构如何演进：看到什么问题？用什么方案解决？

05 AI架构如何落地：先做什么后做什么？各方如何共建？

01

行业趋势与认知

AI 把架构改成什么样子？
业界是怎么做的？

AI 带来的架构变化是什么？——行业与公司，都在巨变

- 6代架构演进主线：单体 → 垂直 → SOA → 微服务 → 云原生 → AI 原生 （每一代架构都源于：业务痛点→技术突破→架构升级）
- AI原生架构驱动力：大模型 + Agent；核心特征：模型为计算单元、Agent 为执行体、上下文为一等公民



架构时代	驱动力	核心特征
单体架构	快速落地	一站式开发，功能叠加后"改一处、影响全局"
垂直架构	业务分化	模块化拆分，跨域协作仍靠硬编码
SOA架构	企业级互联	服务化解耦与复用，集中治理复杂度高
微服务架构	流量井喷	原子自治、独立部署与弹性扩展，运维压力大
云原生架构	极致弹性	容器化、集群化，按量使用、秒级弹性
AI 原生架构	大模型 + Agent	模型为计算单元、Agent 为执行体、上下文为一等公民

行业观点：这是一次计算范式的代际切换，不是一次功能升级

范式切换，不是功能升级——AI增强 vs AI原生

类比：AI 增强 = 燃油车加装辅助驾驶；AI 原生 = 重新设计一辆电车



AI 增强（AI-Enhanced）	AI 原生（AI-Native）
• 思维：在现有系统上「加装」AI	✓ 思维：设计之初把 AI融进架构 DNA
• 架构：传统服务 + AI 调用	✓ 架构：Agent化、动态编排、MCP
• 治理：在原有架构上修修补补	✓ 治理：从根上重新定义「服务」

行业方向已经趋同：Token难预测 / 长任务越权 / Prompt注入——快手都遇到了

- 业界方向：已经趋同，快手真实痛点

●Token 难预测

- 用量算不清、成本不可控

长任务失控 / 越权

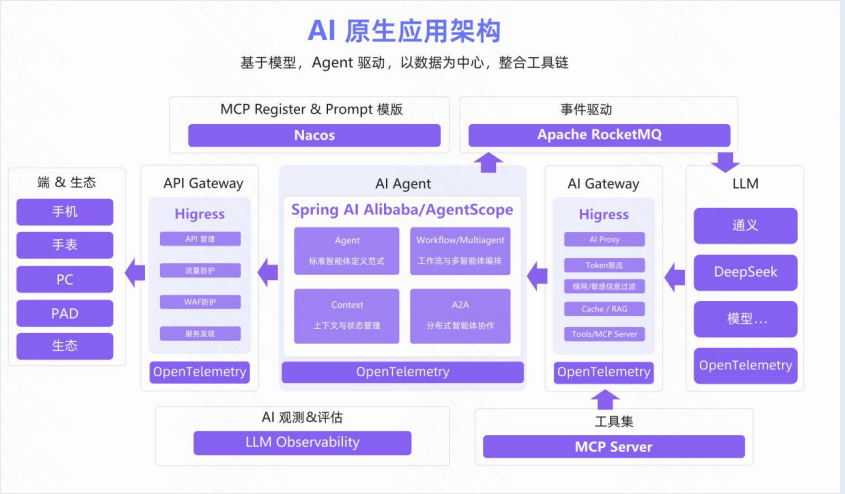
- Agent 跑飞了没人拦得住

Prompt 注入与数据权限穿透

- 一句诱导就能越界拿数据

- 行业实践：阿里云、Spring AI 等，业界在往同一个方向走——AI 网关统一入口、安全护栏、可观测、量化评估

区别只在于：在真实生产里把这套东西跑通



- 政策驱动：网信办《智能体规范应用实施意见》：

•安全

决策权限管理

行为护栏与追溯

行为治理

数字身份

智能体决策

02

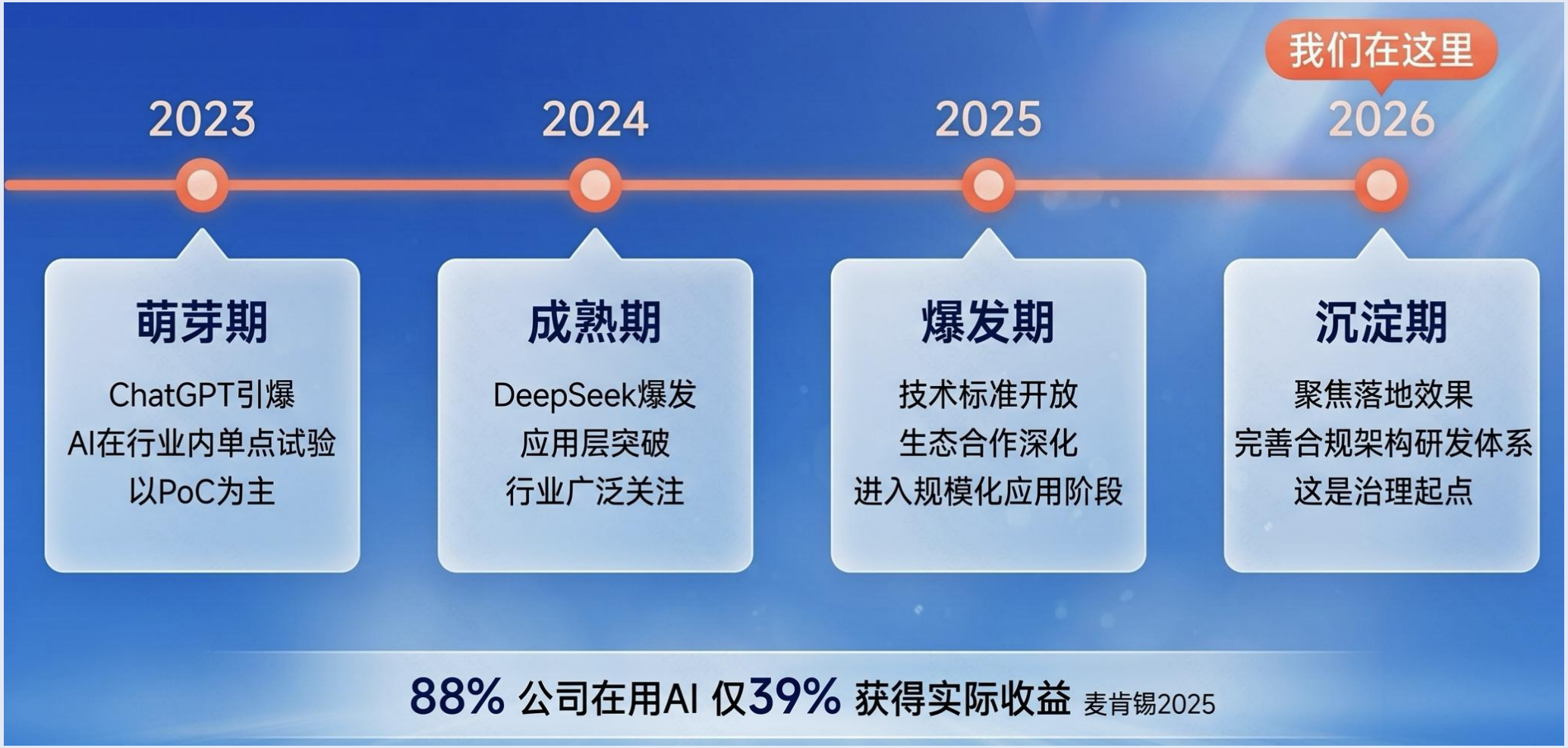
公司现状与挑战

公司遇到什么问题?

挑战在哪?

行业拐点快速到来：已经进入Agent驱动的架构时代

自 25 年初 DeepSeek 爆发，各业务积极落地，公司进入 AI 爆发期



公司AI应用爆发：已经遇到明显的架构问题

自 25 年初 DeepSeek 爆发，各业务积极落地，公司进入 AI 爆发期

30%→60%	1000w→4800w	10000+	30E
AI 代码生成占比 25Q3 → 26Q2	AI 代码生成行数 25Q3 → 26Q2	业务 Agent 应用数 26Q2	Token 单用户 最多月用量

AI 让代码生产速度翻倍，但缺统一规范与底座，问题随之而来

AI 时代的「私搭乱建」与七类真实痛点

私搭乱建：不是哪个团队做得不好，而是 AI 规模化进生产的必经之痛 —— 需要从公司级架构上系统性解决，而非一线各自打补丁。

Case1 类爬虫访问：服务没有 CLI 原生支持，AI 只能像爬虫一样爬页面，频繁触发限流告警

案例一

监控系统GUI截图浪费算力

低效的数据采集方式
占用大量计算资源用于图像处理
而非核心业务逻辑，CPU/GPU负荷过高

案例二

服务缺AI原生接口MCP/CLI

系统难以通过MCP或CLI进行AI自动化集成
缺乏原生标准
传统接口自动化失败

Case2 安全问题：某业务 Agent 通过身份洞穿，调用了本不该访问的数据接口，事后审计才发现

案例三

Skill身份安全洞穿

AI Skill/Plugin缺乏严格身份验证与访问控制
导致权限越权或数据泄露
身份校验失效，敏感数据泄漏

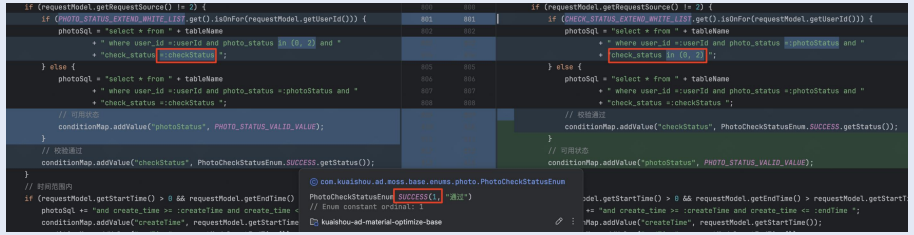
Case3 多 Agent：员工跨 Agent 切换反复登录，身份在各 Agent 间断链，每次都要重新认证

案例四

多Agent各自为战数据不同步

多个独立运作Agent之间缺乏一致状态同步和数据共享机制
导致决策冲突，信息孤岛，状态不一致

Case4 代码问题：AI 生成的代码缺乏对公司架构、性能、稳定性、安全规范的了解，容易写出有问题的代码



Case5 稳定性：模型导致系统不可控，外部模型超时没有降级引起雪崩等问题



Case6 Token 消耗：一个多轮 Agent 任务触发 Token 爆炸，单次消耗超预算 10 倍，事前无人预知

Case7 模型幻觉：78% 的业务没有评测基线，换了模型也不知道效果是变好还是变差

架构挑战：这些问题不做，痛只会升级

最危险的不是某个痛点，而是痛会随规模不断升级

■ 痛会随规模升级：单点之痛 → 系统性风险 → 不可逆的债



问题 / 乱建	不做的直接代价	长期代价
多 Agent 不打通 / **长期技术债**	员工反复登录、Agent 互调身份断链	全局 AI 提效被底层不通卡住，重复造轮子持续浪费
安全没有底线 / **安全隐患**	越权调用、密钥删库、数据泄漏	不敢放手让 AI 进生产；事后补建要重接入 + 回灌审计，代价翻倍
Token效率低 / **成本失控**	Token 用量/归属/需要高效利用治理	规模化不可持续、Token管理困难，业务"不敢用"
代码没有规范 / **AI代码膨胀**	AI 代码"无人能懂"、缺单测兜底	架构加速腐化，缺陷逃逸与故障归因风险上升
模型/稳定无储备 / **稳定性风险**	模型超时/配额耗尽引发依赖服务雪崩	未来能力断档，关键链路不敢承接 AI

03

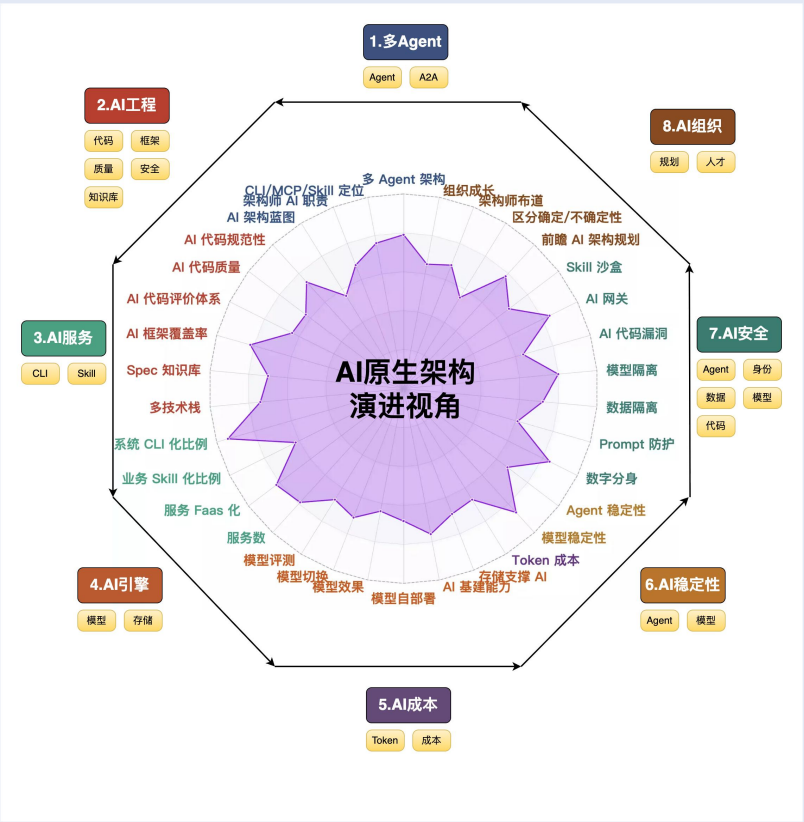
AI架构蓝图规划

AI 原生架构应该怎么建?

公司架构师关注的AI 架构痛点 → 八大视角

- **八大视角**：架构 / 工程 / 服务 / 引擎 / 成本 / 稳定 / 安全 / 组织
- **最痛议题**：多 Agent 打通 95 · CLI 化 95 · 数字分身/Prompt 防护 95 · 代码规范 92 · Token 高效 92

（来源：公司 75 架构师 AI 架构研讨会，47 条真实痛点收敛而来，不是拍脑袋）



视角	痛点议题（研讨紧迫度评分）
架构视角	多 Agent 统一打通与路由编排 95 · Agent 模式定位 90
工程视角	面向 AI 的代码规范 92 · 最佳实践框架 90 · 代码评价体系 88
服务视角	系统 CLI 化 95 · 服务 Skill 化 82
引擎视角	模型评测与选型 88 · 模型平滑切换 85
成本视角	Token 高效利用 92 · 成本指标模型 85
稳定性视角	模型稳定性 85 · Agent 稳定性 78
安全视角	数字分身 95 · Prompt 防护 95 · 数据 / 模型隔离 90
组织视角	架构治理源头规划 88 · AI 视角产品范式 88

补充说明：数字为诉求紧迫度评分（取值 1–100，数字越大表示该议题在架构师研讨追逐中的关注度和推进急迫度越高）

架构师在的 AI 架构演进中的职责

架构师为Agent架构合理性、模型确定性、低成本、稳定性/可用性、安全性负责：

【架构】合理性：Agent/CLI/MCP打通与路由，消除烟囱孤岛

【工程】规范性：规范/框架/评价体系，让AI工程有标准可循

【服务】高效率：面向AI原生的CLI开放体系，提高AI开发效率

【模型】确定性：模型效果的确定性，AI生成代码的正确性

【成本】低成本：低成本使用与衡量，Token治理+成本指标模型

【稳定】可用性：AI服务可用性和调用耗时可控，模型SLA保障

【安全】安全性：解决安全性问题，数字分身+防AI注入+数据与模型隔离/审计

【组织】成长性：前瞻AI架构规划+区分确定/不确定性、架构师布道/组织成长



怎么建之一：AI 原生架构五层技术栈

Agent是AI架构的核心服务，Multi-Agent即是AI时代的多服务架构

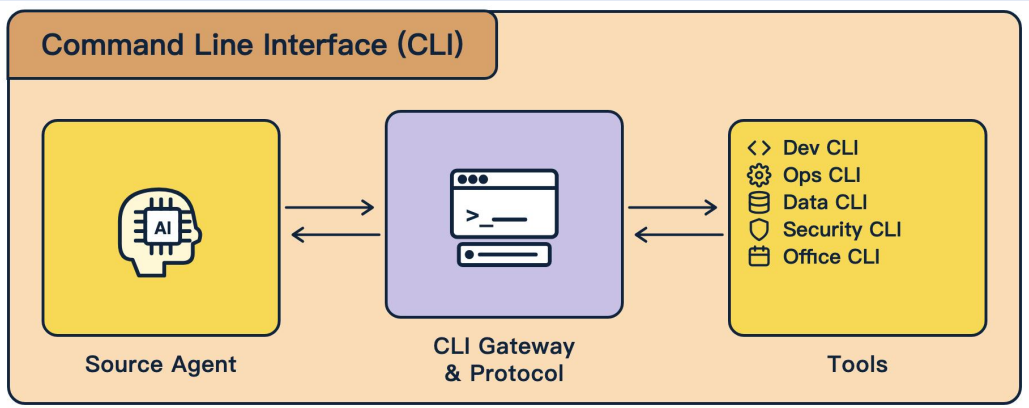
治理层	Agent 运行框架与治理脚手架（Harness、Hermes）
Agent层	LLM 驱动的自主循环系统（MyFlicker、DataAgent）
工具层	可被 Agent 调用的标准化能力单元（Skill / CLI / MCP）
引擎层	模型路由与 Token 分发的调度引擎（万擎）
模型层	底层大模型供给（Claude / GPT / DeepSeek 等）



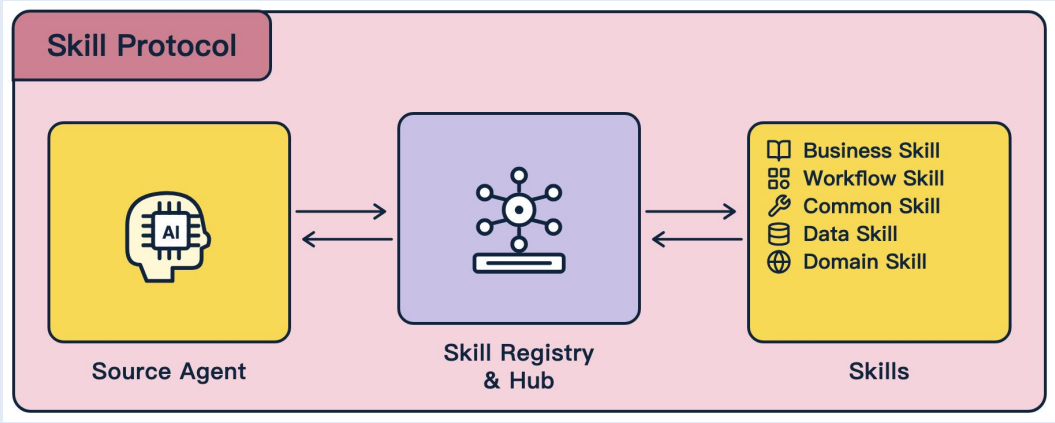
怎么建之二：AI 架构工具层

工具层：需明确CLI、Skill、MCP等各元素在Multi-Agent架构中的定位，形成未来建设的纲要指引

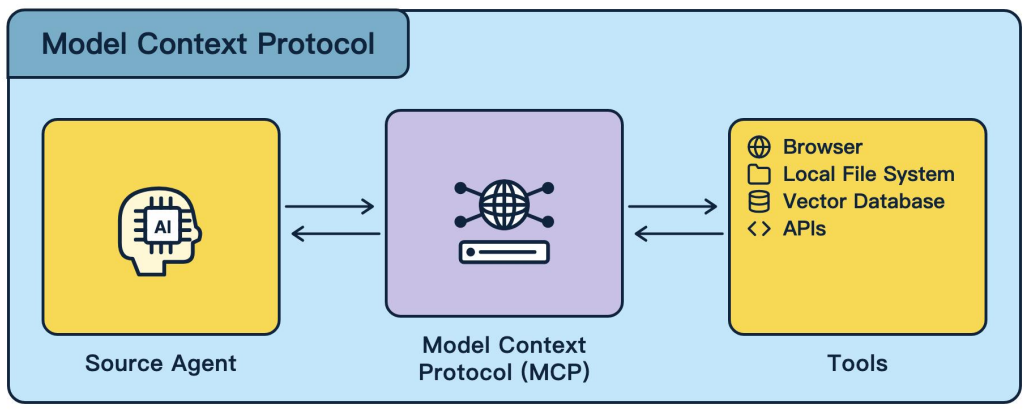
- CLI：面向AI的系统交互接口，适合本地编码与自动化执行



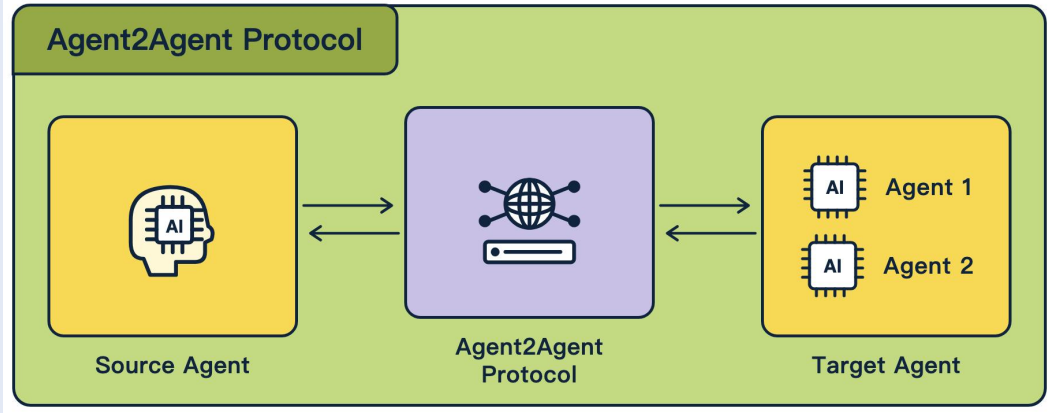
- Skill：封装可复用的原子化能力单元，适合固化特定工作流



- MCP：统一对接外部工具与数据的标准协议，适合跨系统能力调用



- A2A：让Agent成为自治单元的通信协议，适合多智能体协同编排



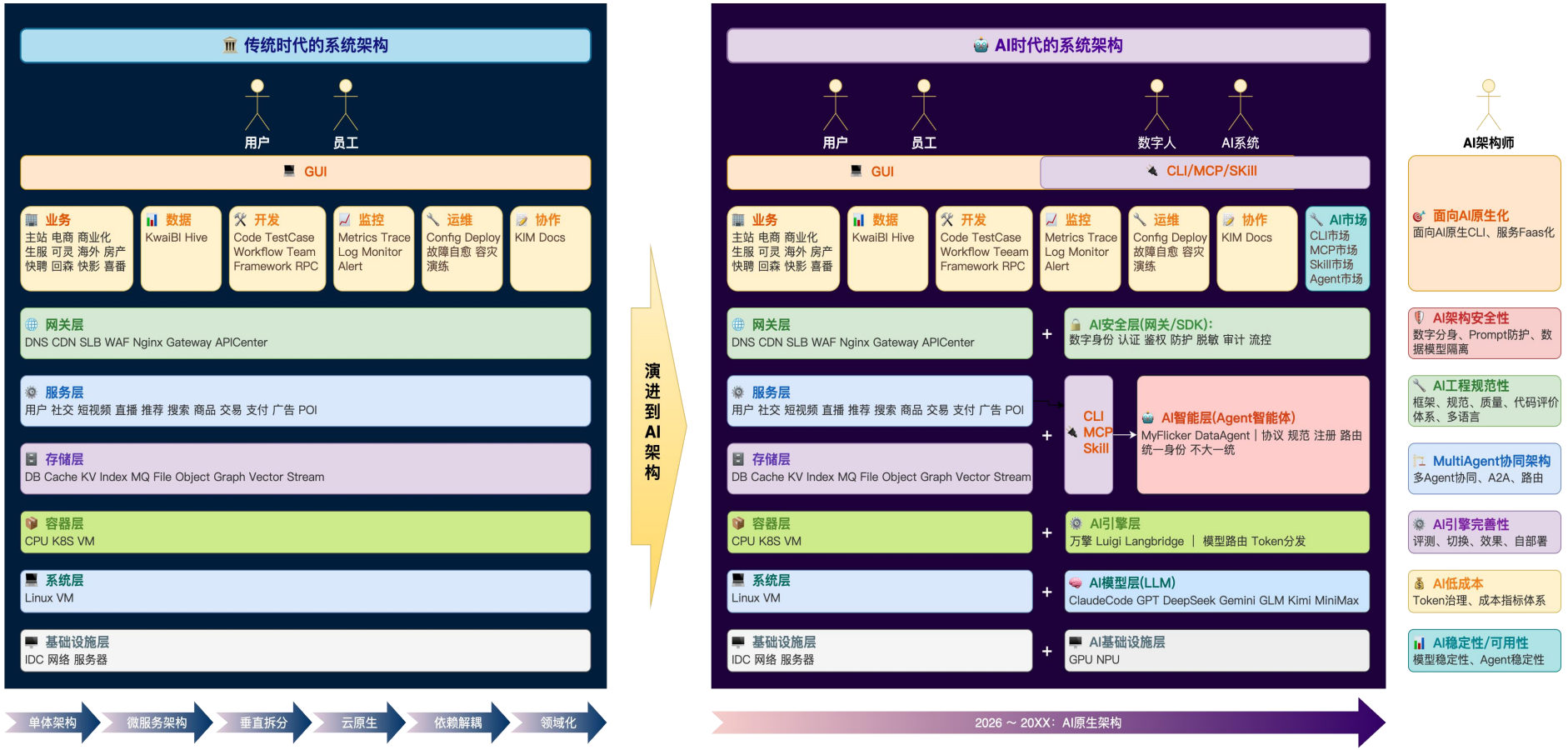
怎么建之三：AI 时代架构设计的八大设计原则

八层各有一条设计原则 —— 合起来就是「AI 友好」的判断基准

架构 · 合理性 改造前：多 Agent 反复登录、身份断链 → 改造后：A2A 统一身份，全链路打通	**工程 · 规范性** 改造前：AI 代码无规范、出问题无人能懂 → 改造后：Skill 门禁拦截，问题在提交前暴露	**服务 · 高效率** 改造前：AI 像爬虫爬接口，失败率高 → 改造后：CLI 标准化调用，覆盖率 >99%	**模型 · 确定性** 改造前：业务无评测基线 → 改造后：一键评测 + 最优模型
打通Agent消除孤岛	AI代码规范/框架/评价	服务CLI 开放体系	模型效果与生码正确
Token · 低成本 改造前：Token 用量/归属不清 → 改造后：可观测、可归因、ROI 可度量	**稳定 · 可用性** 改造前：模型超时无降级，引发雪崩 → 改造后：统一网关限流	**安全 · 安全性** 改造前：身份洞穿越权、无审计 → 改造后：数字身份 + 防护 + 全程可追溯	**组织 · 成长性** 改造前：各团队各自摸索 → 改造后：统一蓝图 + 全员Skill课程
Token 治理+成本模型	Agent / 模型 SLA 保障	数字身份鉴权+防注入+隔离	AI架构前瞻规划+AI布道成长

怎么建之四：AI 时代架构蓝图什么样

AI原生并非抛弃传统架构，而是在现有架构基础上，以LLM和Agent为中心来思考服务架构的构建和运行




AI时代的系统架构
用户
员工
数字人
AI系统
GUI
CLI/MCP/SKill
业务
主站 电商 商业化
生服 可灵 海外 房产
快聘 回森 快影 喜番
数据
KwaiBI Hive
开发
Code TestCase
Workflow Teeam
Framework RPC
监控
Metrics Trace
Log Monitor
Alert
运维
Config Deploy
故障自愈 容灾
演练
协作
KIM Docs
AI市场
CLI市场
MCP市场
Skill市场
Agent市场
网关层
DNS CDN SLB WAF Nginx Gateway APICenter
AI安全层(网关/SDK):
数字身份 认证 鉴权 防护 脱敏 审计 流控
服务层
用户 社交 短视频 直播 推荐 搜索 商品 交易 支付 广告 POI
CLI
MCP
Skill
AI智能层(Agent智能体)
MyFlicker DataAgent | 协议 规范 注册 路由
统一身份 不大一统
存储层
DB Cache KV Index MQ File Object Graph Vector Stream
容器层
CPU K8S VM
AI引擎层
万擎 Luigi Langbridge | 模型路由 Token分发
系统层
Linux VM
AI模型层(LLM)
ClaudeCode GPT DeepSeek Gemini GLM Kimi MiniMax
基础设施层
IDC 网络 服务器
AI基础设施层
GPU NPU
AI架构师
面向AI原生化
面向AI原生CLI、服务Faas化
AI架构安全性
数字分身、Prompt防护、数据模型隔离
AI工程规范性
框架、规范、质量、代码评价体系、多语言
MultiAgent协同架构
多Agent协同、A2A、路由
AI引擎完善性
评测、切换、效果、自部署
AI低成本
Token治理、成本指标体系
AI稳定性/可用性
模型稳定性、Agent稳定性

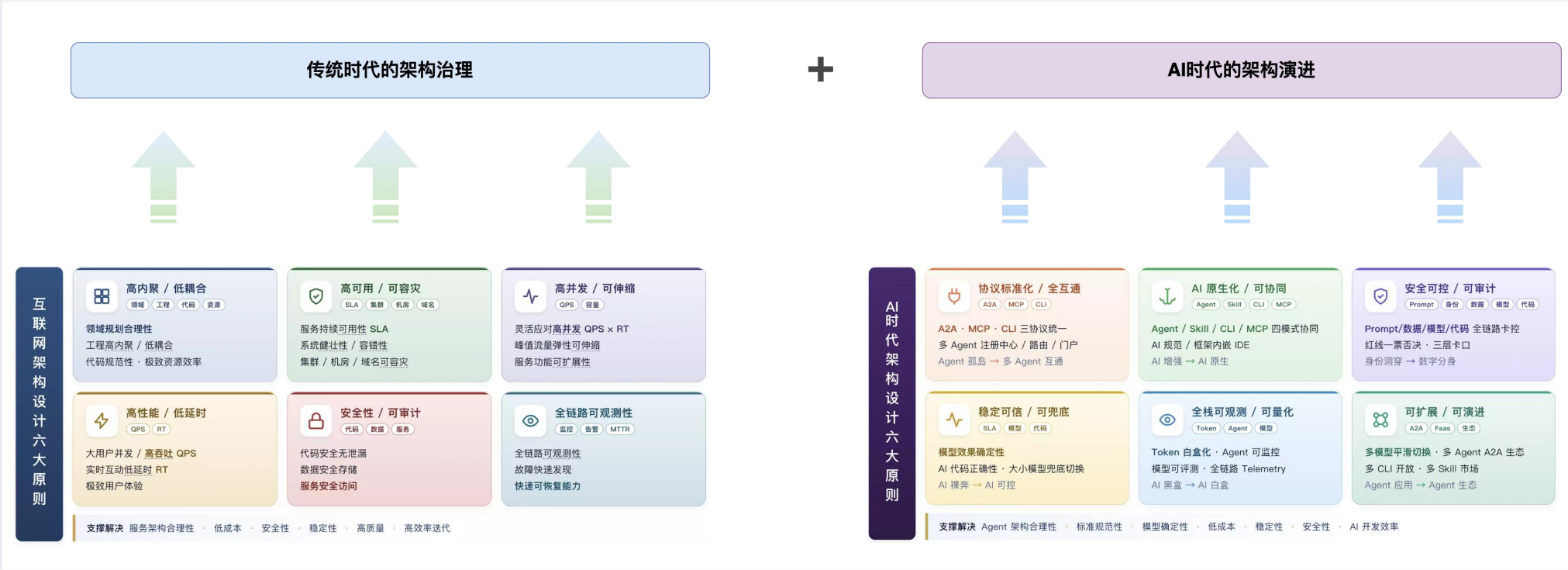
AI架构与传统架构的关系：AI时代架构 = 传统架构 + 智能体

AI时代架构 和 传统时代架构 两者不是取代关系，而是共存且演进关系（你中有我、我中有你）：AI时代架构 = 传统架构 + AI智能体



设计原则对比：AI时代架构 vs 传统时代架构

两者架构设计原则和围绕的核心目标：高可用性 vs 智能化可控性



04

AI架构如何演进

看到什么问题?

用什么方案解?

八视角打法总览：从问题 → 到打法

架构视角（重点介绍）

多 Agent 统一打通与路由编排
（身份 SSO / 门户路由 / 统一底表 / Agent · Skill 规范 / Skill 化 · A2A / MCP · SDK 协议）

工程视角（重点介绍）

AI 代码规范 / 框架 / 评价体系
（强制规范底座 + 单测兜底线 + AI 接入脚手架）

服务视角（重点介绍）

全面 CLI 化、Skill 化、FaaS
（CLI 已 KO / Skill 收口 MySkill / 长期 FaaS）

引擎视角

模型评测 / 平滑切换 / 自部署
（统一评测标准 + 评测集市场 + 一键评测 + 季度清单）

Token成本视角

Token 高效利用 + 成本模型
（可观测→可归因→高效用→ROI 评价）

稳定视角

模型 / Agent 稳定性与可观测
（模型 SLA + 统一网关/SDK + 长任务可观测）

安全视角

身份 4A / Prompt 防护 / 数据隔离
（统一 Agent ID / 注入检测 / 四级数据分级 + 出域网关）

组织视角

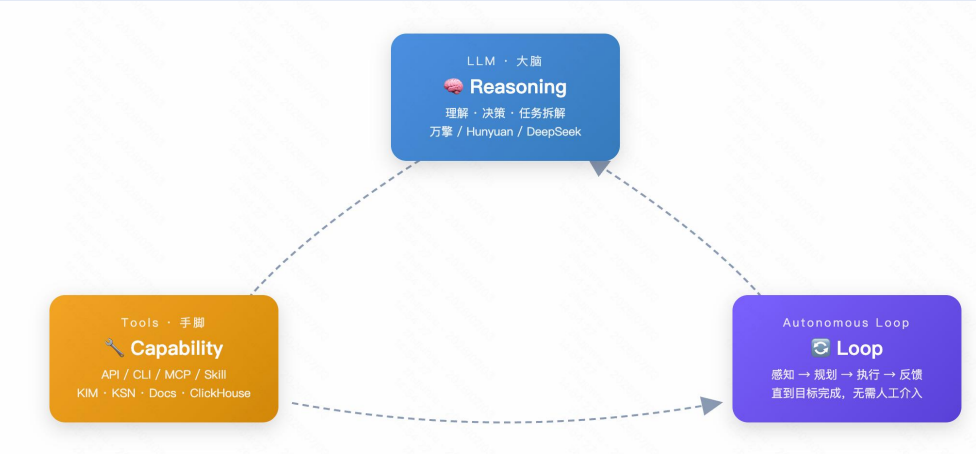
前瞻规划 + 架构师布道
（架构 + 11 条业务线 / 定期研讨机制）

1、架构视角： Multi-Agent身份定义/分类/身份互通性

① 定义清楚什么是Agent：单体、工作流？拉齐全公司对智能体的定义，未来才好治理

Agent（智能体）是由 LLM 驱动、能够自主感知环境、制定计划并持续采取行动，直至完成指定目标的智能系统。

公式化表达（沿用 Anthropic 在 *agentic coding tool* 中的定义）：**Agent = LLM（大脑） + Tools（手脚） + Autonomous Loop（自主循环）**



概念	是否是 Agent
Prompt 模板 / 单次问答	
Skill	
MCP Server	
CLI 工具	
RAG 应用（仅检索 + 单次问答）	
Claude Code / CodeFlicker / OpenClaw / Jarvis / DataAgent/ArchAgent/ langbridgeClaw/ kwaiClaw	
定义前：5w+	定义后：1w+

1、架构视角： Multi-Agent身份定义/分类/身份互通性

② 建立分类标准：对公司1万Agent分类。不同的Agent要求身份和安全性不同

从四个维度对Agent进行分类：

- 部署维度：本地部署 / 云端部署 / 平台托管
- 能力维度：平台型 / 领域型
- 员工/使用视角分类（数字员工、数字分身）
- 开发模式（单体、配置化）



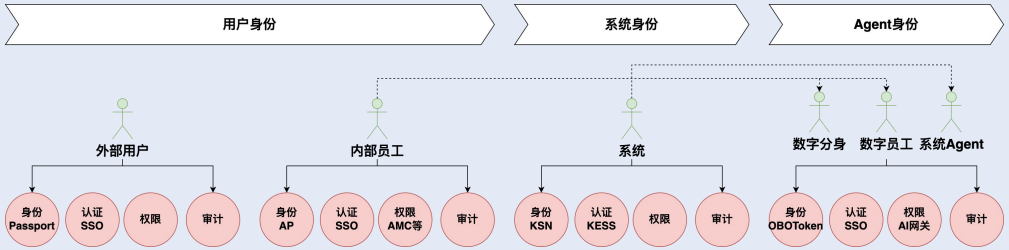
定义后分类：

	部署模式		能力维度		身份视角分类			模式分类	
	本地	云端	通用型	领域型	数字分身	数字员工	系统 Agent	独立开发	平台配置
myflicker									
DataAgent									
ArchAgent									
kim分身									
OpenClaw									

1、架构视角： Multi-Agent身份定义/分类/身份互通性

③ 对Agent进行认证、鉴权、互通身份

■ 系统架构中扩展新的身份：



■ 认定一个 Agent 的身份，本质上是要依次回答四个环环相扣的问题：

✓ **Agent 是谁**

✓ **Agent 代表谁**

✓ **Agent 能做什么**

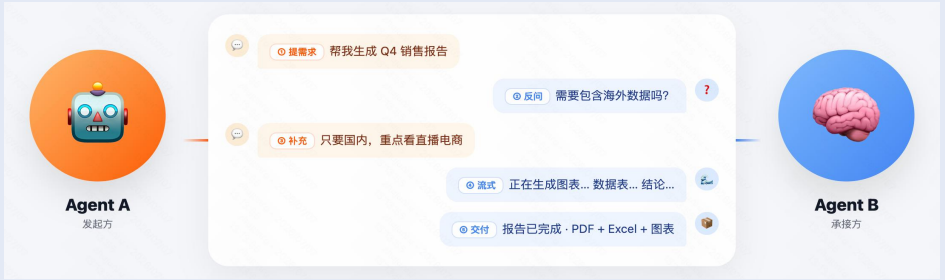
✓ **Agent 做过什么**



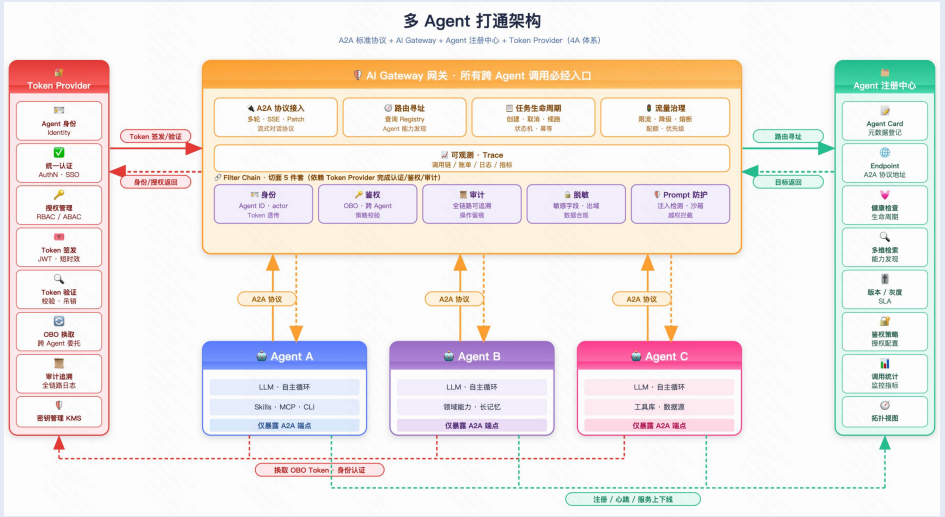
改造

密从不

■ Agent身份互通场景：



■ Agent身份互通架构：

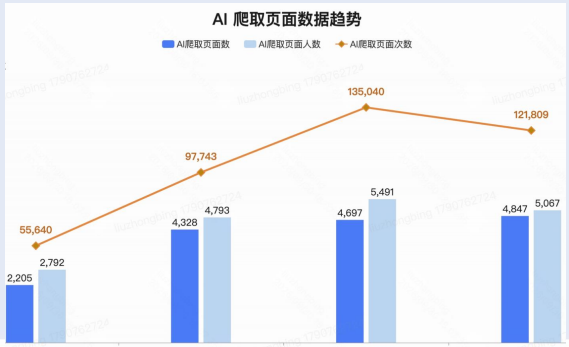


2、服务视角：全面 CLI 标准化访问API、Skill 标准化框架

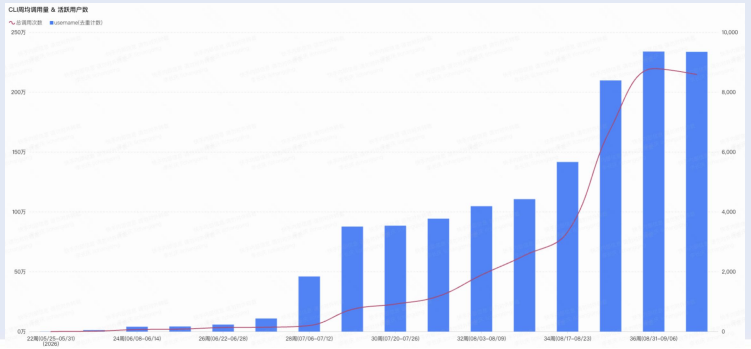
CLI标准化：建设统一的CLI标准化能力，抵御AI爬虫式访问内部系统



✓ 改造后：AI类爬虫访问得到抑制：



✓ 改造后：标准化CLI访问量大幅度上涨：



Skill标准化：10+市场、1w+Skill，建设约束脚手架

业务线	名称	Skill数
A业务	MySkill	7000+
	skill 广场	700+
B业务	takumi skill 广场	200+
	langbridge 广场	800+
	rc-skill	90+
	Skill市场	60+
C业务	skills	500+
	skill 广场	700+
D业务	mario	700+
E业务	skill合集共享	100+

Skill脚手架/约束：

- Manifest 规范
- 目录结构规范
- 命名规范
- 文档 & 描述规范
- 安全规范（强约束）
- 可观测规范

3、工程视角：知识 + 规范 + 评价 + 需求可验收 + 交付可部署

- **强制规范**：公司级 AI 基础规范作底座（命名/分层/接口契约/日志/异常/安全编码，强制约束非建议），业务可继承叠加专属规范；单测必须覆盖兜底逻辑（限流/熔断/降级/超时/重试）作为上线门槛（准入+准出+自动化测试三道闸）
- **标准框架API**：公司级「AI 接入脚手架」自动对接 RPC/OpenAPI/Hive/Kafka/KDB/Redis，业务开箱即用；规范本质：降低出错概率，而非提升代码质量上限

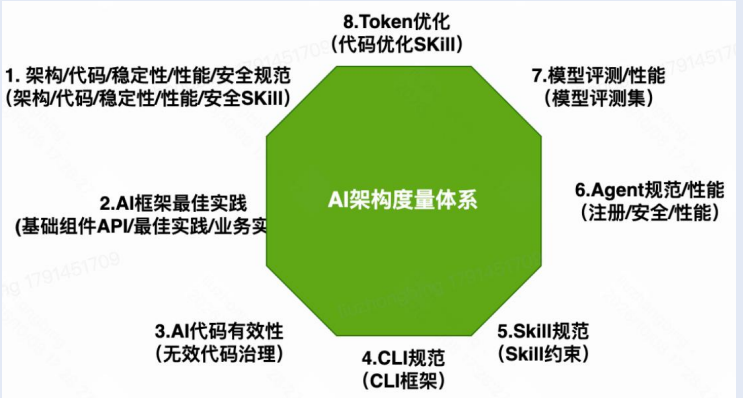
AI代码规范约束：架构/安全/质量



AI代码最佳实践：框架/客户端



AI代码评价体系：有效性/正确性



3、工程视角：建设Skill规范门禁，对AICoding进行约束

门禁不是建议文档，是嵌在研发流程里、会真的拦住提交的那道闸门
落地效果：各业务Harness工具均置入Skill门禁系统，拦截数千AICoding问题

组件知识SKill

① 组件标准化门禁

基础组件 API：统一 maven 依赖、规范版本、实例构造统一

· 解决 AI 调用中间件时的准确性与可靠性问题

架构规范SKill

② 架构与业务规约门禁

· 架构规范：框架 / 依赖 / 分层 / 调用契约 / 防腐

· 业务领域知识：业务术语、数据模型、调用约定

· 业务最佳实践 Skill

稳定性规约Skill

③ 稳定性 / 性能 / 安全门禁

· 编码规范：Java / Python / Go / C++ / 前端 / 移动端

· 稳定性：超时 / 降级 / 熔断；性能：慢查 / 缓存 / 分页

· 安全：数据 / 权限 / 鉴权漏洞；质量：单测 / 容量 / 耗时

Agent性能Skill

④ Agent 使用门禁

· Agent 安全 / 性能

· 模型性能

· Token 优化

门禁自己也要被评测

· Skill 存在多处开发/多态引用问题
· → 设计多态开发与继承机制，规则集复用与分层

规则分层复用与多态继承

· 共 18 个评测
197 条规则、1806 个测试用例

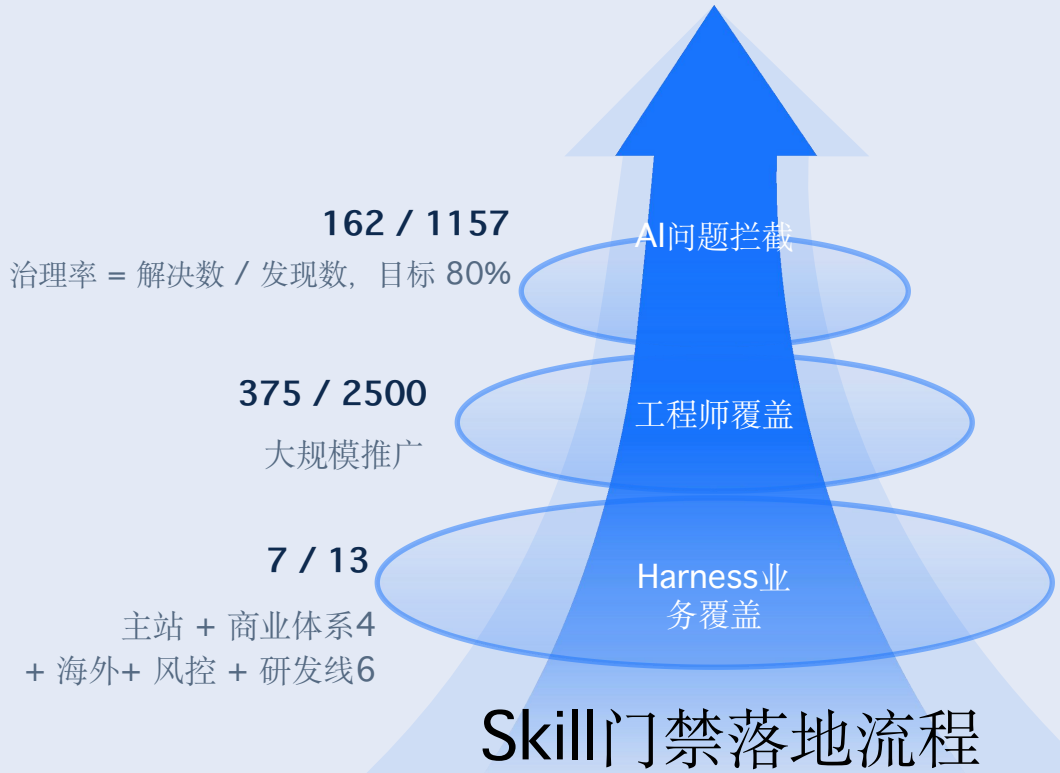
评测覆盖度 100%

· 不通过多为边界 Case
· 如父类 getValid() + 子类 isValid()，仅分析子类漏了父类

通过率 100%

3、工程视角：Skill门禁嵌入Harness，实现对AI的研发流程管控

拦截数不是终点，治理率才是 —— 试点结果：发现 1157 个，解决 162 个，AI 辅助解决占 14%



AI 健康测评体系发现了什么？

发现 1157 个问题：

- ✓ 编码规范 37%（缺注解、硬编码 Token）
- ✓ 稳定性 30%（缺灰度开关、缺打点）

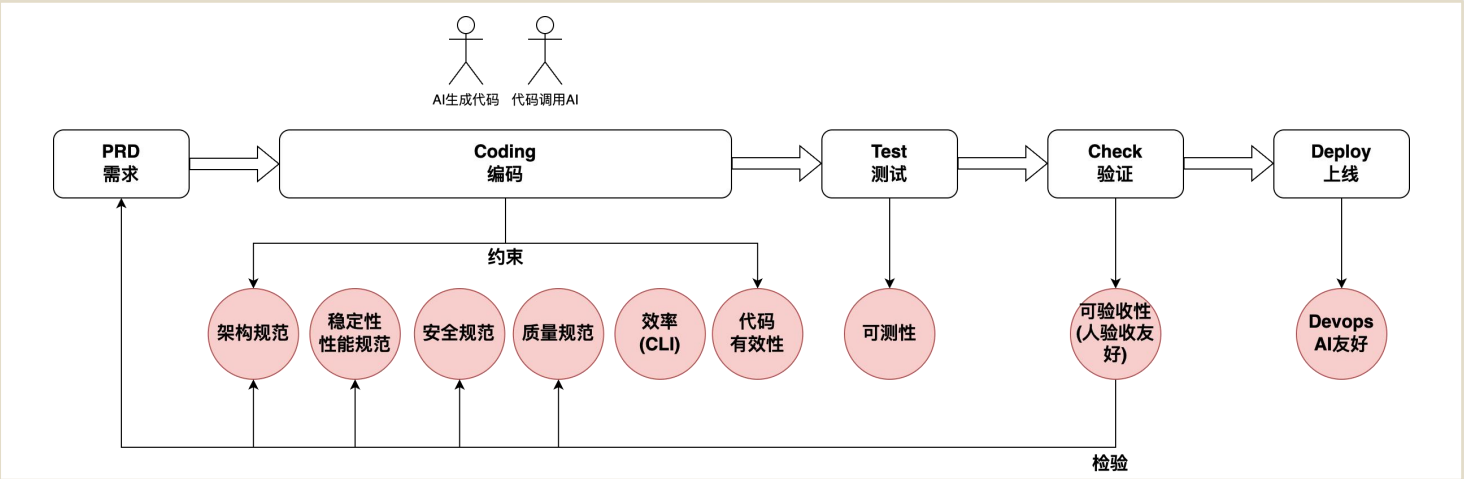
已治理 162 个：

- ✓ 架构 40%（无防腐层、缺 NPE 处理、 if-else 嵌套 >4 层）
- ✓ 稳定性 30%

其中 14% 由 AI 辅助直接解决；

3、工程视角：让 AI 的交付可被验证

AICoding可控性规范环节：



AICoding后的可验证闭环：AI 写完代码不算交付，能编译、能部署、能验证才算



代码编译

流水线、编译、打包成可部署产物

环境部署

容器/物理机部署、域名、Nginx 路由、Kafka/RocketMQ topic、Kbus consumer

功能验证

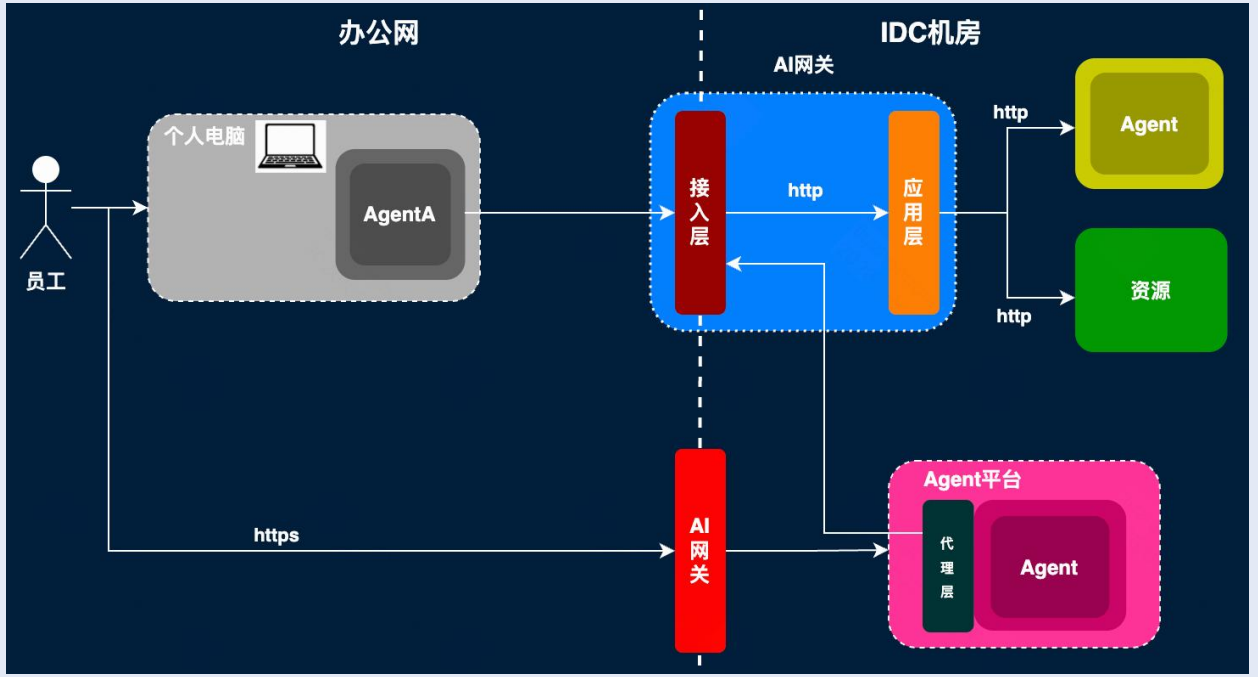
API/RPC 接口测试、Task 触发、消息收发、监控与日志

4、安全架构视角：Agent身份鉴权 + 环境/数据/模型隔离 → 守住红线

安全红线：

- 数字身份鉴权（统一 Agent ID / aaid / 独立虚拟账号 / 最小必要授权）
- Prompt 注入越权泄漏防护与审计
- 数据/模型隔离
- 落地效果：改造前发生过 Agent 越权调用、密钥误操作、数据越界访问；改造后每个 Agent 有数字身份，每次调用可追溯到自然人，越权行为在网关层被直接拦截





5、Token视角： Token 高效利用 + 归因模型

解决问题：Token 用量 / 归属 / ROI 不清；GPU+Token 时代不再适用 CPU 时代成本核算

- **先可观测**（说得清）：建立 ROI 评价体系（产出=价值，Token 消耗=成本）
- **再高效用**（管得好+用得省）：建「用户/业务视角」的 Token 微观可观测与可归因
  - **线上系统使用TokenROI提效**：模型ROI测评和选型指引，缓存命中率优化，批量推理
  - **员工使用TokenROI评价**：线上多轮对话压缩和拆分，模型智能路由，Skill Token消耗优化



6、稳定视角：解决Agent/模型可观测性，并进行稳定性约束

解决：业务代码访问模型 / Agent稳定性可观测性问题



7、引擎视角：面向未来的能力储备

解决：模型评测与选型、模型平滑切换与升级、面向AI的统一基础设施、面向AI的存储基建能力、模型的自部署能力

模型评测与选型：	模型平滑切换与升级：	面向AI的统一基础设施：	面向AI的存储基建能力：	模型的供给/自部署：
业务**不知道该不该换模型**、平台的新能力没法在业务铺开	模型的**切换成本太高**，不敢切、切得不及时	AI应用的快速孵化、各个Agent需要统一的基础设施**支持提高效率**	存储要面向AI能力支持，**向量数据、记忆、多模态**	**外部模型不稳定**、开源好模型在公司内需要**快速部署**



05

AI架构如何落地

先做什么后做什么?

各方如何共建?

先做什么后做什么——价值判断三标准：不按声量，按价值

总体导向：聚焦全公司共同的基础性问题，优先夯实公共地基 —— 地基稳固，上层 AI 价值方能充分释放



共性优先

多业务普遍面临的共性问题，优先于单一团队的个性化诉求 —— 公司统一解决一次，避免重复建设

底线优先

触及安全、稳定底线，或阻碍全局 AI 提效的问题，优先解决

价值可量化

收益可用成本节省、效率提升、风险规避衡量的事项，优先推进 —— 价值清晰，自然形成共识

按四档觉得优先级：自底向上夯地基

第一档

守底线（必须当期）

多 Agent 打通、安全底线

第二档

控成本（同步推进）

Token 高效利用、AI 成本模型

第三档

水到渠成（顺势就做）

全面 CLI 化、AI 代码规范/框架/评价

第四档

长期储备（稳步推进）

模型评测/切换/自部署、模型与 Agent 稳定性、FaaS、AI 架构蓝图



各方需要做什么？——需要全员一起共建

- **架构横向角色**：需求分析与规划组织方（包括架构治理、安全、PMO项目、通道规范等），负责前期的问题澄清和规划调研，及项目过程中的跟踪核对、组织方案对齐、组织落地改造、效果跟进等；
- **建设方角色**：Agent/CLI/模型建设方（包括引擎/模型、各种Agent工具建设房等），重点对基建能力进行设计和建设；
- **业务方角色**：评审、使用和改造方（所有业务线AI应用场景），重点参与方案评审、使用落地、Agent改造等；



有哪些基建发布？——规范落地到Skill集成到业务Harness流程

产出一系列：标准、AI规范、框架、Skill工具、最佳实践、AI评价体系、红线、知识库等

视角	打法重点	架构横向角色（架构治理/委员会/通道/安全/PMO/运营/HR）										
		标准	规范	框架	工具	最佳实践	评价体系	红线	认证	知识库	课程	文章
		《AI标准白皮书》	《AI架构规范》	《标准框架》	《AI工具》	《最佳实践》	《AI评价体系》	《AI红线》	《市场认证》	《知识库/MD》	《AI架构课程》	《KStack宣发》
		《AI原生架构白皮书》							架构治理/安全		《AI原生架构规划》	《AI架构往哪走？》
1.架构视角	重点规划Agent通用能力沉淀打通 快速明确Agent架构模式定位 明确架构师职责	MultiAgent建设标准	Agent&A2A规范	Agent框架		Agent开发实践			Agent官方认证	Agents知识库	架构专场 （Agent 模式定位 / 职责边界 / 蓝图路线图）	《多Agent打通》
		未来AI架构蓝图(6月)										《AI蓝图规划》
2.工程视角	重点建设AI代码规范/最佳实践框架 中期建设代码/质量评价体系和知识库 长期解决技术债的AI化治理		AI代码/测试/安全规范		规范SpecMD			代码红线			工程专场 （AI 规范落地姿势 / 评价体系 / 敢放手指数）	《AI代码规范》
				AI开发框架	AI脚手架	AI最佳实践指引						《AI框架/最佳实践》
							AI代码如何评价					《AI代码评价体系》
										AI知识库		《AI知识库》
						AI回测能力						《AI代码质量如何兜底》
						AI拆分架构						《AI重构》
3.服务视角	重点解决全面CLI化 长期思考Faas化		CLI规范/命名空间	CLI开发框架	CLI工具				CLI官方认证	CLI知识库	服务专场 （CLI 化规范 / Skill 化路径 / FaaS 探索）	《CLI系列》
			SKill规范						SKill官方认证	SKill知识库		《SKill系列》
4.引擎视角	长期提供模型评测/切换/效果/部署能力										引擎专场 （评测选型 / 切换升级 / 存储支撑 AI）	《模型评测》
												《模型平滑》
												《模型效果》
												《模型自部署》
												《AI基建规划》
												《AI存储基建规划》
5.成本视角	重点解决Token高效利用问题 长期建立AI时代下成本指标体系		Token规范				Token度量体系	成本红线			成本专场 （Token 治理 / ROI 评价 / 成本健康分）	《Token系列》
							AI成本评价体系					
6.稳定性视角	长期解决模型/Agent稳定性可观测性						稳定性指标	稳定性红线			稳定性专场 （模型放心调 / Agent 放心跑 / 稳定性总分）	《AI稳定性系列》
												《AI模型工程最佳实践》
7.安全视角	重点解决数字身份鉴权/Prompt防护/数据隔离 长期解决AI分析漏洞							安全红线			安全专场 （数字分身鉴权 / Prompt 防护 / 数据隔离 / Skill 沙盒）	《AI安全解决方案》
						AI分析漏洞						

总结与回顾

1 行业趋势与认知

- 6代架构演进主线
- AI增强 vs AI原生
- 行业对AI风险认知

2 公司现状与痛点

- 行业拐点在26年
- 公司5万AI应用爆发
- AI私搭乱建的案例
- 架构痛点与挑战

3 AI架构蓝图规划

- 8大视角打法
- 架构师在AI中的职责
- AI原生架构5层技术栈
- 工具CLI/Skill/MCP/Agent
- AI架构八大设计原则
- AI时代架构蓝图
- AI架构/传统架构关系

4 AI架构如何演进

- 架构:定义/分类/协议
- 服务:全面CLI化
- 工程:规范/评价/框架
- 安全:Agent数字身份
- 成本:TokenROI/归因
- 稳定:Agent/模型观测
- 引擎:评测/切换/部署

5 AI架构如何落地

- 先做什么后做什么
- 分四档优先级
- 各方需要做什么?
- 有哪些基建发布?

先夯地基，业务才能在 AI 时代跑得更快、更稳、更远!

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

欢迎带着AI最佳实践，一起交流AI架构演进：

- 订阅AI架构治理实战分享，关注公众号「快手技术」
- 参加27.1 Harness技术沙龙，加入「快手技术沙龙」
- 26.12 AICon，发布《快手 AI 原生架构演进白皮书》

快手技术AI干货	Harness技术沙龙	AICon发布白皮书
		